Programming Language Principles, Design, and Implementation

# Lecture 5 - Big-Step Operational Semantics

Sean Moss

University of Birmingham

# Where are we?

- **Part 1: Principles of programming**
  - Functional languages
  - **Operational semantics**
  - Type systems
  - Abstract datatypes
- Part 2: Compilers

# Today

**Big-step operational semantics**

- Operational semantics as transition systems
- Evaluation strategies
- Implementation in Haskell — exercise

# Recap: the untyped $\lambda$-calculus

**Syntax**:

$$M ::= x \mid \lambda x.M \mid M\ M$$

**$\beta$-equivalence** (no execution order)

| | |
|---|---|
| $\beta$-conversion | $(\lambda x.M)\ N =_\beta M[x\backslash N]$ |
| reflexivity | $M =_\beta M$ |
| symmetry | if $M =_\beta N$ then $N =_\beta M$ |
| transitivity | if ($M =_\beta N$ and $N =_\beta P$) then $M =_\beta P$ |
| left-apply | if $M =_\beta N$ then $M\ P =_\beta N\ P$ |
| right-apply | if $M =_\beta N$ then $P\ M =_\beta P\ N$ |
| lambda | if $M =_\beta N$ then $\lambda x.M =_\beta \lambda x.N$ |

**$\beta$-reduction**

$$(\lambda x.M)N \mapsto_\beta M[x\backslash N]$$

plus the last three ('congruence') rules.

# Operational semantics

**Goal:** given the syntax of a programming language, one has to describe **how to compute** with the language.

**Approaches**:

- **Denotational semantics**

  The meaning of a term is an abstract mathematical object (a denotation)

  Laws to reason about program behaviours can be immediately derived from properties of the mathematical objects

- **Axiomatic semantics**

  The meaning of programs is provided by laws (axioms) about their behaviour

  Focuses on the process of reasoning about programs

- **Operational semantics** (covered in this lecture)

  Specifies the behaviour of programs by describing how they execute on abstract machines that perform steps of computations

  Provides a link to implementations

# Operational semantics

**Structural**: rules are specified in a syntax directed way

Several kinds:

- **Small-step operational semantics** (covered next lecture)

  Specifies how programs execute 1 step at a time

  Easier to model and reason about complex features such as concurrency (where intermediate steps are critical)

- **Big-step operational semantics** (covered in this lecture)

  Specifies how programs execute in 1 step, from the initial configuration to the final value

  Also called **natural semantics** (because of its resemblance with Natural Deduction)

  Closely models a recursive interpreter

  Sometimes leads to smaller proofs (less rules)

# Transition system

The operational semantics of a language is provided by a **transition system** that consists of

- a set of **states**
- a binary "next state" **transition relation** between states

**Examples:**

- In the case of the untyped $\lambda$-calculus, states can simply be $\lambda$-terms
- In a language with references, states can be pairs of a program and a store (storing the values currently help in the memory cells).

The transition relation can be defined as a **proof system** consisting of **derivation rules** in a Natural Deduction style.

There is a transition from $M$ to $N$ if there is a derivation (a proof tree) that $M$ and $N$ are related.

# Big-step semantics of the untyped $\lambda$-calculus

Specified by a **transition system**

- **states** are $\lambda$-terms
- **transitions** are provided by 2 inference rules

A big-step semantics specifies how terms compute to values in 1 step.

**Values**: $\lambda$-abstractions of the form $\lambda x.M$.

**1st rule**:

$$\frac{}{\lambda x.M \Longrightarrow_v \lambda x.M}\ \textsc{val}$$

**2nd rule**:

$$\frac{M \Longrightarrow_v \lambda x.P \quad N \Longrightarrow_v Q \quad P[x\backslash Q] \Longrightarrow_v V}{M\ N \Longrightarrow_v V}\ \textsc{app}$$

**Note:** $M \Longrightarrow_v N$ only when $N$ is a $\lambda$-abstraction.

**Note:** Reductions are not allowed inside $\lambda$s

**Note:** We will explain the $v$ subscript later in the lecture

# Big-step semantics of the untyped $\lambda$-calculus

Booleans:

$$\begin{array}{lcllcl} \texttt{T} & = & \lambda x.\lambda y.x & \texttt{and} & = & \lambda a.\lambda b.a\ b\ \texttt{F} \\ \texttt{F} & = & \lambda x.\lambda y.y & \texttt{or} & = & \lambda a.\lambda b.a\ \texttt{T}\ b \\ & & & \texttt{not} & = & \lambda a.a\ \texttt{F}\ \texttt{T} \end{array}$$

What does $(\texttt{or}\ \texttt{T}\ \texttt{F})$ compute to according to this semantics?

$$\dfrac{\dfrac{\dfrac{}{\texttt{or} \Longrightarrow_v \texttt{or}}\ \textsc{val} \quad \dfrac{}{\texttt{T} \Longrightarrow_v \texttt{T}}\ \textsc{val} \quad \dfrac{}{\lambda b.\texttt{T}\ \texttt{T}\ b \Longrightarrow_v \lambda b.\texttt{T}\ \texttt{T}\ b}\ \textsc{val}}{\texttt{or}\ \texttt{T} \Longrightarrow_v \lambda b.\texttt{T}\ \texttt{T}\ b}\ \textsc{app} \quad \dfrac{}{\texttt{F} \Longrightarrow_v \texttt{F}}\ \textsc{val} \quad \dfrac{?}{\texttt{T}\ \texttt{T}\ \texttt{F} \Longrightarrow_v \texttt{T}}\ \textsc{app}}{\texttt{or}\ \texttt{T}\ \texttt{F} \Longrightarrow_v \texttt{T}}\ \textsc{app}$$

$$\dfrac{\dfrac{\dfrac{}{\texttt{T} \Longrightarrow_v \texttt{T}}\ \textsc{val} \quad \dfrac{}{\texttt{T} \Longrightarrow_v \texttt{T}}\ \textsc{val} \quad \dfrac{}{\lambda y.\texttt{T} \Longrightarrow_v \lambda y.\texttt{T}}\ \textsc{val}}{\texttt{T}\ \texttt{T} \Longrightarrow_v \lambda y.\texttt{T}}\ \textsc{app} \quad \dfrac{}{\texttt{F} \Longrightarrow_v \texttt{F}}\ \textsc{val} \quad \dfrac{}{\texttt{T} \Longrightarrow_v \texttt{T}}\ \textsc{val}}{\texttt{T}\ \texttt{T}\ \texttt{F} \Longrightarrow_v \texttt{T}}\ \textsc{app}$$

# Big-step semantics of the untyped $\lambda$-calculus

Let $\Delta = \lambda x.xx$

What does $(\Delta\Delta)$ compute to according to this semantics?

$$\dfrac{\dfrac{}{\Delta \Longrightarrow_v \Delta}\ \text{VAL} \qquad \dfrac{}{\Delta \Longrightarrow_v \Delta}\ \text{VAL} \qquad \dfrac{\vdots}{\Delta\Delta \Longrightarrow_v}}{\Delta\Delta \Longrightarrow_v}\ \text{APP}$$

We are back to the same term

$(\Delta\Delta)$ does not compute to a value!

# Big-step semantics of the untyped $\lambda$-calculus

Let $\Delta = \lambda x.xx$

What does $((\lambda x.\lambda y.y)(\Delta\Delta))$ compute to according to this semantics?

$$\dfrac{\dfrac{}{\lambda x.\lambda y.y \Longrightarrow_v \lambda x.\lambda y.y}\ \textsc{val} \quad \dfrac{\vdots}{\Delta\Delta \Longrightarrow_v} \quad \dfrac{}{\Longrightarrow_v}}{(\lambda x.\lambda y.y)(\Delta\Delta) \Longrightarrow_v}\ \textsc{app}$$

As before this term does not compute to a value!

**Note**: if we reduce the top $\beta$-redex first, we obtain $\lambda y.y$

# Evaluation strategies

The above big-step operational semantics requires evaluating the argument before doing a $\beta$-reduction.

I.e., before $\beta$-reducing $(\lambda x.M)N$, the argument $N$ must be first computed to a value.

The above semantics specifies a particular **evaluation strategy** called **call-by-value** (or **strict**), where arguments to functions are always evaluated whether or not they are used in the body of the function.

In general $\lambda$-terms have several redexes (e.g. $(\lambda x.x)((\lambda y.y)(\lambda z.z))$ has $2$). An evaluation strategy **fixes the order** in which redexes must be reduced.

The above semantics is a: **call-by-value big-step operational semantics**.

In contrast, the **call-by-name** evaluation strategy (also called **lazy**) only evaluate arguments when they are actually used.

# Call-by-name big-step semantics of the untyped $\lambda$-calculus

The **call-by-name big-step operational semantics of the untyped $\lambda$-calculus** also has 2 rules:

**1st rule**:

$$\frac{}{\lambda x.M \Longrightarrow_n \lambda x.M}\ \texttt{VAL}$$

**2nd rule**:

$$\frac{M \Longrightarrow_n \lambda x.P \quad P[x\backslash N] \Longrightarrow_n V}{M\ N \Longrightarrow_n V}\ \texttt{APP}$$

**Note**: once again, $M \Longrightarrow_n N$ only when $N$ is a $\lambda$-abstraction.

**Notation**:

- **call-by-value**: $M \Longrightarrow_v N$
- **call-by-name**: $M \Longrightarrow_n N$

# Call-by-name big-step semantics of the untyped $\lambda$-calculus

Let $\Delta = \lambda x.xx$

Does $\Delta\Delta$ compute to a value using a

- call-by-value strategy? No
- call-by-name strategy? No

What does $((\lambda x.\lambda y.y)(\Delta\Delta))$ compute to according to the call-by-name semantics?

$$\dfrac{\dfrac{}{\lambda x.\lambda y.y \Longrightarrow_n \lambda x.\lambda y.y}\,\textsc{val} \qquad \dfrac{}{\lambda y.y \Longrightarrow_n \lambda y.y}\,\textsc{val}}{(\lambda x.\lambda y.y)(\Delta\Delta) \Longrightarrow_n \lambda y.y}\,\textsc{app}$$

# Properties

## Values

- If $M \Longrightarrow_v N$ then $N$ is a $\lambda$-abstraction
- If $M \Longrightarrow_n N$ then $N$ is a $\lambda$-abstraction

## Deterministic

- If $M \Longrightarrow_v N$ and $M \Longrightarrow_v P$ then $N = P$
- If $M \Longrightarrow_n N$ and $M \Longrightarrow_n P$ then $N = P$

## $\beta$-equality

- If $M \Longrightarrow_v N$ then $M =_\beta N$
- If $M \Longrightarrow_n N$ then $M =_\beta N$

# Implementation of an interpreter in Haskell

Big-step operational semantics provide specifications for implementing interpreters.

**Demo**

# Conclusion

## What did we cover today?

- Call-by-value big-step operational semantics of the untyped $\lambda$-calculus
- Call-by-name big-step operational semantics of the untyped $\lambda$-calculus
- Implementation of a Haskell call-by-value interpreter — exercise.

## Next time?

- Small-step operational semantics
- Normalisation